

5/19/20
Today:

Part 1: C++ memory fundamental.

1) Variable: int $x = 10$

x
memory location which stores value 10.

2) Address: x

~~for~~

↓

x ↓

10

memory.

3) Point

int: datatype, which affects:

- how the value is interpreted
- which operations are valid.
- how much storage is required.
- how the compiler generates machine instructions.

so, $\text{int } x = 10$ & $\text{double } x = 10;$

2) Pointer:

~~array~~ $\text{int } *p = \&x;$

x → contains the integer 10.

p → address of x .

Here; pointer declaration: $\text{int } *p$

It means: $\text{int } *$ x is an integer.

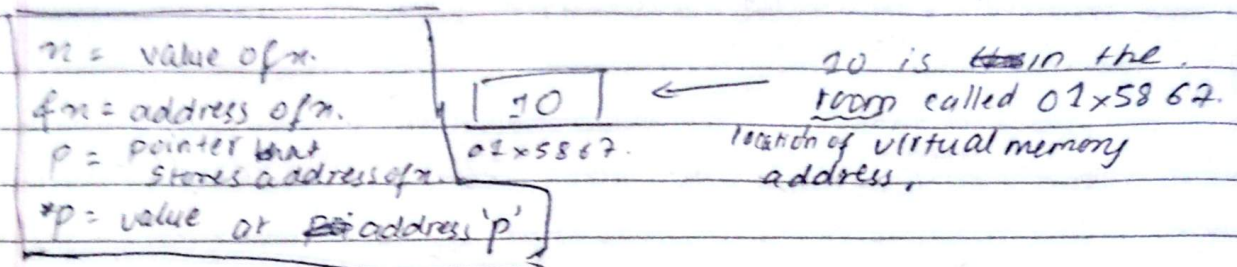
$\text{int } *p.$

p : p is a pointer that point to an integer object.

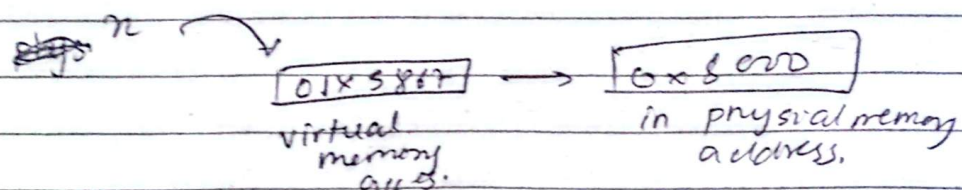
p is a sort of variable which stores the address of an integer object.

$*$ → pointer type declaration.

*p → go to the address of p & access the value there.



Virtual memory address: It gives the illusion that the computer has massive, private and continuous block of RAM.



- It's given by Operating System, we will study it later.

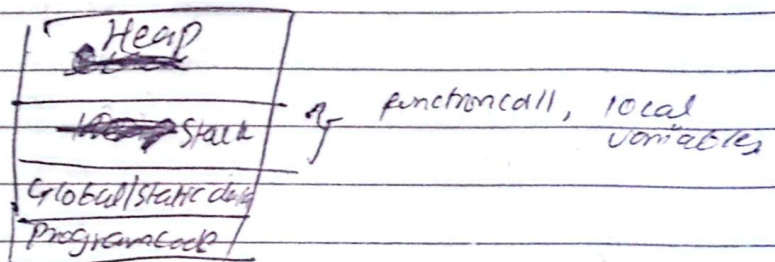


Fig: memory layout.

Stack vs heap

a) Program Code:

- It stores the compiled code of our program.
- It is only read-only to prevent the accidental modification.

n) Global static data.

- It stores global and static variables, which have lifetime ^{till} entire program.

c) Stack Memory:

- It is temporary work space for the function execution.
- Last-In-First-Out (LIFO)
- It is automatically controlled by compiler.

d) Heap memory.

- This is the portion which you can interact with the memory location.
- O.S. tracks messy heap memory by using 'free list'.
- When we request for the memory (using malloc in C and new in C++), first of all the manager needs to find an empty memory according to the request & it provides. So, it is more slower.
- We ~~also~~ need to clean it by using, (free() in C or delete in C++) manually. But, it is automatically cleaned up by Garbage collector in Java, Python, C#.

⑥

Why does modern C++ uses smart pointer/RAII instead of new/delete?

⇒

we need to do all things manually in new/delete. But using smart pointer/RAII will automatically delete after work is completed.

part 2- Computer Architecture.

1) Central Processing Unit (CPU):

- It executes instructions.

C++: $n = 10 + 20.$

- It doesn't go directly to the CPU ~~for~~ to add 10 & 20.

C++ program $\rightarrow$ Compiler $\rightarrow$ machine (code) ^{instruction}
 $\downarrow$
 data moves b/w CPU & memory. $\leftarrow$ CPU execute machine code.

The fundamental components of CPU.

- Registers
- ALU
- Control units.

2) Registers:

- Register is the fast and small storage location inside the CPU.
- It stores data temporarily so that CPU can ~~st~~ calculate it fast.

$\left\{ \begin{array}{l} S+3: - \text{Take } S \text{ from memory to register (A)} \\ \quad - \text{Take } 3 \text{ from memory to } \text{another register (B)} \\ \quad - \text{Add the value by ALU} \\ \quad - \text{Store } 8 \text{ in registers (C).} \end{array} \right\}$

So, registers are working space of CPU.

For example:

- Hard Disk $\rightarrow$ cupboard to store the books.
- ~~Reg~~ RAM $\rightarrow$ study table where we store take few books.
- Registers $\rightarrow$ study area where we keep books which we are currently studying in front of our hand.

② why not RAM instead?

$\rightarrow$ Registers are faster, coz they are inside the CPU.

4) Arithmetic Logic Unit (ALU).

- Part of the CPU which performs calculation and logical decision. It is the calculator of the CPU.

Register A		ALU		Register B.
5	$\Rightarrow$	+	$\Rightarrow$	8
3				

5) Machine code [Instructions].

~~2 = a + b~~ $\rightarrow$ Computer doesn't understand. ^{compiler}
 over code directly, so ~~it~~ convert it into machine instructions defined by ISA [Instruction Set Architecture].

Something like:

```
LOAD R1, a
LOAD R2, b
ADD R3, a R1, R2
STORE x, R3
```

- ISA is the rules that the software uses to communicate with a CPU.

Examples of ISA:

- x86-64: Intel/AMD PCs.
- ARM: phone, embedded system.
- RISC-V: open resource used for education, and research.

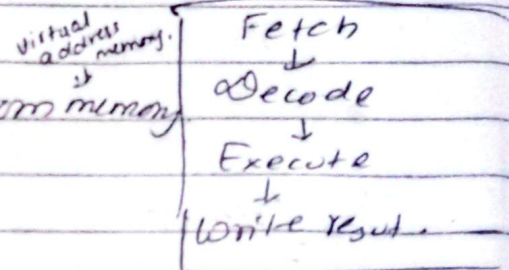
④ Instruction Cycle.

→ the process a CPU to execute one instruction.

a) Fetch:

- CPU gets the instruction from memory.

memory.	Instruction
1000	ADD R1, R2
1004	SUB R3, R4



fetch → 1000, then, fetch 1004
like that.

b) Decode

- ~~decode~~ ^{understand} the instruction.
- Control Unit identifies:
 - what operation to perform.
 - which data registers are needed

c) Execute

- CPU performs the actual operation.

ALU ⇒ $5 + 3 = 8$

d) Write Result:

- the result is stored in a register or memory.

7) ISA



↳ is like a menu: 'what?'

ADD : add two numbers

SUB : subtract two numbers.

LOAD : bring data from memory.

8) Microarchitecture → like a kitchen.

↳ It executes those ISA instructions.

It handles how it is converted.